

Assignment 2B — Retrieval-Augmented Generation (RAG) Pipeline

Chunking · Dense & Sparse Retrieval · Reranking · Tabular RAG · 20 Marks

Prerequisite: Complete Assignment 1A or 1B. You will reuse the cleaned domain .txt corpus from Assignment 1 Part A directly as your RAG document store. No new data collection is required. Your fine-tuned adapter from Assignment 1 Part B is the RAG generator.

Component	Part	Marks
Document Processing & Chunking Strategies	Part A	6
Retrieval: Dense vs Sparse vs Hybrid	Part B	7
Reranking & Tabular RAG	Part C	7
TOTAL		20

Completing this assignment gives you a production-grade RAG pipeline on your enterprise domain corpus — the same retrieval infrastructure used by enterprise AI assistants at scale. Each part escalates: you chunk → retrieve → rerank → handle structured data.

PART A — Document Processing & Chunking Strategies [6 Marks]

Load your cleaned domain .txt files from Assignment 1 Part A. Apply three chunking strategies and measure chunk quality metrics for each.

Step A1 — Implement Three Chunking Strategies [3 Marks]

- Fixed-Size
- Sliding Window
- Semantic Chunking

Use Max tokens = 200, overlap percentage = 10%, for semantic chunking (eg. Preserve the sentence boundaries).

Step A2 — Chunking Quality Analysis [3 Marks]

For each strategy, compute the following metrics and fill the table:

Strategy	Total Chunks	Avg Chunk Size (words)	Std Dev Size	Broken Sentences (%)
Fixed-Size	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>
Sliding Window	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>
Semantic	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>

PART B — Retrieval: Dense vs Sparse vs Hybrid [7 Marks]

Use your best chunk strategy from Part A. Build three retrieval systems and benchmark them on 10 domain queries from your Assignment 1 evaluation set.

Step B1 — Dense Retrieval [3 Marks]

- **Embedder** — Embed all chunks and mention your embedding configuration (model name, vector dimensions, similarity metric used, batch size if applicable).
- **Index** — Build FAISS IndexFlatIP index or any other index; retrieve top-5 chunks per query
- **Benchmark** — Report index build time, query latency (ms), and top-1 chunk relevance

Step B2 — BM25 Sparse Retrieval + Hybrid Fusion [4 Marks]

- **BM25** — Build BM25 index; retrieve top-5 chunks per query
- **Hybrid** — Hybrid fusion: combine BM25 and dense scores using Reciprocal Rank Fusion (RRF) formula
- **Comparison** — Fill benchmark table across all 3 retrieval methods for 10 queries

Retrieval Method	Avg Query Latency (ms)	Top-1 Relevance (avg 1-3)	Top-3 Coverage
Dense	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>
Sparse	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>
Hybrid	<i>[fill]</i>	<i>[fill]</i>	<i>[fill]</i>

Relevance Scale: Score each top-1 retrieved chunk manually on a 1–3 scale:

1 = Not relevant (chunk is off-topic or factually unrelated to the query)

2 = Partially relevant (chunk touches the topic but misses key information)

3 = Highly relevant (chunk directly answers or strongly supports the query)

PART C — Reranking & Tabular RAG [7 Marks]

Step C1 — Cross-Encoder Reranking [3 Marks]

Add a cross-encoder reranking stage after your best first-stage retriever from Part B. Compare top-3 results before and after reranking for 10 queries.

- **Metrics** — Measure: reranking latency per query; rank change rate (% of queries where rank-1 changed)
- **Analysis** — Report whether reranking improves top-1 relevance for your domain (manually verify 5 queries)

Step C2 — Tabular RAG [4 Marks]

Extract at least 3 domain tables from your corpus (In case tables were not available in the original corpus, use new pdfs having tables). Build a tabular RAG system that can answer structured queries from table content.

- **Extraction** — Extract tables from PDFs using camelot or pdfplumber; save as CSV
- **Serialisation** — Serialise each table row as text: 'Column1: val1 | Column2: val2 | ...'
- **Indexing** — Index serialised rows alongside text chunks previously used
- **Demo** — Demonstrate 3 tabular queries where retrieved table rows enable precise answers

Write 100 words: for your domain, when does tabular RAG outperform text-chunk RAG? Provide at least one concrete example from your results.

Submission Deliverables

Please submit all the relevant .ipynb and corresponding html file with outputs, and the tables_chunks.csv (Serialised table rows from domain PDFs).